



UNIVERSITÀ
DEGLI STUDI
DI PADOVA

DIPARTIMENTO DI MATEMATICA

LAUREA TRIENNALE IN INFORMATICA

Biblioteca Virtuale in C++

Aldo Bettega

Indice

1	Introduzione	2
2	Descrizione del modello	2
3	Polimorfismo	4
	GUI	4
	Observer pattern	4
	Json	4
	Visitor Pattern	4
4	Persistenza dei dati	5
5	Funzionalità aggiuntive	5
6	Rendicontazione ore	5

1 Introduzione

La Biblioteca Virtuale offre un sistema di gestione di vari media, imitando lo stile e organizzazione di MLOL, una Media Library Online. La biblioteca offre una pagina utente dalla quale è possibile visualizzare i tipi di media disponibili con un' intuitiva e pulita interfaccia grafica. Nella sezione admin, pensata per gli addetti ai lavori, sarà invece possibile manipolare i vari articoli.

2 Descrizione del modello

Nel modello logico è presente una struttura gerarchica di 8 classi: 4 classi astratte e 4 classi concrete. La classe base è biblioteca che contiene gli attributi comuni di un media bibliotecario. Da questa derivano le classi Multimedia e Media cartaceo, la prima vuole rappresentare tutti gli articoli fruibili attraverso mezzi tecnologici, come film o cd, mentre la seconda articoli cartacei in formato di libro, giornale o rivista. Ho riflettuto a lungo per quanto riguarda l'estendibilità di questo modello e in che parte della gerarchia posizionare i singoli attributi. Inizialmente avevo una classe derivata da Biblioteca chiamata Media fisico che avrebbe dovuto rappresentare tutti gli articoli che potessero essere posseduti in modo fisico e quindi prenotati o restituiti. Tale classe aveva gli attributi copieTotali, copieInPrestito, disponibile e lingua poichè ogni articolo fisico nel suo formato presenta una lingua specifica: banalmente nei libri la lingua in cui è scritto, mentre nei media video la lingua sulla copertina. La classe media fisico era stata pensata per potenziare l'estendibilità del progetto con la possibilità di aggiungere risorse digitali con evidenti differenze da articoli fisici, tuttavia in corso d'opera per mantenere una gerarchia semplice è stato necessario unire la classe media fisico con biblioteca. Un'altra scelta che ha migliorato l'estendibilità del progetto è stata spostare il tipo "autore" dalla classe biblioteca a gerarchie più basse specializzandolo in "regista", "artista" e "autore". Questo perchè lasciarlo troppo in alto nella gerarchia avrebbe generalizzato troppo il concetto e ci sarebbero state incoerenze con tipi concreti come "periodico" che non ha un vero e proprio autore ma è frutto di un insieme di articoli scritti da diversi autori. Discorso simile per "editore" e "casa di produzione" poichè nella lingua italiana si fatica a trovare un termine che racchiuda il concetto per tutti i tipi di articoli, per questo ho scelto di delegare questo tipo a classi più basse nella gerarchia specializzandolo ed evitando fraintendimenti lessicali. Per dimostrare l'estendibilità della gerarchia si potrebbero aggiungere altri tipi di articoli come "serie tv", derivato dalla classe concreta "media video", con attributi "numero episodi, numero stagioni e numero dvd". Poi anche "audiolibro" derivante da "media audio" con attributi "narratore, libro originale", oppure "fumetto" derivante da "libro" con attributi "nome saga, numero volume, disegnatore e is colorato".



Figura 1: Model

3 Polimorfismo

Il polimorfismo è stato ampiamente usato sia nella parte grafica che nella parte di persistenza dei dati.

GUI

Nella parte grafica in particolare ho voluto applicare il principio DRY (don't repeat yourself) della programmazione ad oggetti per avere codice più pulito, maggiore manutenibilità e ripetizione minima del codice. Ad esempio dalla classe base astratta `basePage` derivano `userPage` e `adminPage` che hanno quasi tutta la parte grafica in comune sennonché in `adminPage` sono presenti i bottoni aggiungi, salva e carica, discorso molto simile per `baseDetailPage` da cui derivano `adminDetailPage` e `userDetailPage`. Un altro caso più complesso in cui ho avuto maggiori difficoltà a livello di implementazione è stato il `baseEditWidget`, una classe base astratta che racchiude le funzionalità di creazione e aggiornamento di un articolo. Ho voluto racchiudere queste due operazioni in una sola classe notando la loro somiglianza nell'interfaccia grafica, con la differenza che in edit i campi del form sono già compilati in attesa di essere modificati. Questa classe viene poi implementata dalle classi concrete `CdEditWidget`, `LibroEditWidget`, `PeriodicoEditWidget` e `MediaVideoEditWidget` che rappresentano i form degli articoli concreti. Ognuno per creare il form corretto richiama dalla classe base i metodi corretti sfruttando la gerarchia del modello logico: un libro ha come antenati media cartaceo e biblioteca, dunque chiamerà i metodi di creazione di queste due sezioni e poi si occuperà di lavorare sui suoi campi specifici.

Observer pattern

Ho inoltre implementato l'observer pattern in modo tale che le due pagine admin e user si aggiornino quando vengono salvate delle modifiche nella biblioteca e vengano fatte vedere graficamente.

Json

Per quanto riguarda la persistenza nel `jsonManager` sono presenti metodi di save che effettuano un dispatch sul tipo da salvare e viene scelta l'implementazione corretta, in modo che l'oggetto sia salvato in formato json con i campi corretti.

Visitor Pattern

Viene poi implementato il visitor pattern che sfrutta il polimorfismo per creare all'interno della detail page la visualizzazione corretta del dettaglio dell'articolo. Viene infatti chiamato il metodo `visit` che effettua un dispatch automatico sulla base del tipo dinamico, trovata l'implementazione corretta viene eseguito uno static cast (il quale si è sicuri di poterlo fare poiché è basato sulla gerarchia modello logico) che richiama il visit padre: in questo modo se voglio creare il dettaglio di un cd avrò una catena di chiamate `cd->media audio->multimedia->biblioteca` che si occuperà di disporre a schermo nel modo corretto le informazioni dell'articolo.

4 Persistenza dei dati

La logica relativa a questa parte viene gestita dalla classe `jsonManager` che si occupa di caricare, salvare, eliminare e aggiornare i dati della biblioteca, agisce da ponte tra la `mainWindow` e i dati in formato json, occupandosi inoltre della gestione degli observer. Le funzioni di load servono per caricare da un file json i dati necessari e metterli in una `QList<Biblioteca*>` utilizzabile dall'interfaccia grafica. Il loader principale legge il campo classe e si occupa di chiamare il loader della classe concreta, il quale poi chiamerà prima i loader antenati nella gerarchia del modello, poi si occuperà dei suoi campi specifici, ritornando un puntatore polimorfo all'oggetto creato, inseribile nella lista `QList<Biblioteca*>`. Questi loader sfruttano strutture dati presenti nel .h per il trasferimento e conversione dei dati che verranno poi usate nella creazione dell'oggetto concreto. Le funzioni di save invece si occupano del procedimento opposto: prendono un oggetto dalla lista `QList<Biblioteca*>` e lo salvano in un `JsonObject` riempiendo opportunamente i campi json.

5 Funzionalità aggiuntive

Ho diviso l'interfaccia in due pagine principali: l'area admin e l'area utente. In questo modo gli utenti potranno eseguire operazioni sicure, come visualizzare, cercare, prenotare e restituire media. Mentre solo un addetto ai lavori, accedendo dall'area admin, potrà apportare modifiche alla biblioteca, aggiungendo, modificando ed eliminando i media. Un media bibliotecario potrà essere prenotato solo se risulta essere disponibile, se questa operazione andrà a buon fine il numero degli articoli disponibili decremerà in modo dinamico. Se si è in possesso di un media bibliotecario, questo potrà essere restituito, con il conseguente incremento dei media disponibili. Al momento della restituzione di un articolo ne verrà registrata la fruizione, in questo modo l'utente potrà vedere quali articoli ha già letto, ascoltato o visto. Tra le funzionalità è inoltre presente un panel che contiene diversi filtri di ricerca, come la ricerca per autore o per lingua. Per far funzionare al meglio la ricerca attraverso filtri, sarà necessario prima resettare la ricerca affinché vengano ripuliti i campi precedentemente usati. I diversi media sono inoltre suddivisi graficamente in vetrine orizzontali per tipologia, queste vetrine si aggiornano dinamicamente in base alla ricerca.

6 Rendicontazione ore

Attività	Ore Previste	Ore Effettive
Studio e progettazione	5	5
Sviluppo del codice del modello	5	5
Studio del framework Qt	10	10
Sviluppo del codice della GUI	25	30
Test e debug	10	11
Stesura della relazione	3	3
totale	58	64

Per lo sviluppo della GUI ci è voluto più tempo del previsto, poichè scrivendo codice mi sono reso conto di possibili miglioramenti che hanno reso più complessa la logica della struttura ma rispettavano maggiormente le best practices della programmazione ad oggetti. Un esempio è il `baseEditWidget` da cui derivano gli edit concreti.